

## Тема: Unit-тестування асинхронного коду з Jest

**Мета:** набуття практичних навичок написання автомати-зованих тестів для асинхронного TypeScript-коду з використанням фреймворку Jest, зокрема: • налаштування Jest з TypeScript (ts-jest); • написання unit-тестів для асинхронних функцій; • використання матчерів Jest (toBe, toEqual, toThrow, toHaveBeenCalledTimes тощо); • тестування Promise-based коду через async/await та .resolves/.rejects; • використання lifecycle hooks (beforeEach, afterEach) та fake timers (jest.useFakeTimers); • вимірювання покриття коду (code coverage); • застосування generics у TypeScript для типізації тестів.

### Хід роботи:

#### 3.1. Завдання 1: Типізація функцій через generics

```
import { delay } from './delay';

export async function retryOperation<T>({
  operation: () => Promise<T>,
  maxRetries: number = 3
}): Promise<T> {
  let lastError: unknown;
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      console.log(`Спроба ${attempt}...`);
      return await operation();
    } catch (error) {
      lastError = error;
      if (attempt < maxRetries) {
        await delay(100);
      }
    }
  }
}
```

|           |      |               |        |      |                                                 |                   |      |         |
|-----------|------|---------------|--------|------|-------------------------------------------------|-------------------|------|---------|
|           |      |               |        |      | ДУ «Житомирська політехніка».26.121.8.000 – Лр2 |                   |      |         |
| Змн.      | Арк. | № докум.      | Підпис | Дата |                                                 |                   |      |         |
| Розроб.   |      | Ігнатюк А.М.  |        |      | Звіт з<br>лабораторної роботи                   | Літ.              | Арк. | Аркушів |
| Перевір.  |      | Фуріхата Д.В. |        |      |                                                 |                   | 1    | 10      |
| Керівник  |      |               |        |      |                                                 | ФІКТ Гр. ІПЗ-23-1 |      |         |
| Н. контр. |      |               |        |      |                                                 |                   |      |         |
| Зав. каф. |      |               |        |      |                                                 |                   |      |         |

```

    }
  }
  throw lastError;
}

```

```

export async function processInBatches<T, U>(
  items: T[],
  batchSize: number,
  processor: (batch: T[]) => Promise<U[]>
): Promise<U[]> {
  try {
    const results: U[] = [];
    const total = Math.ceil(items.length / batchSize);

    for (let i = 0; i < items.length; i += batchSize) {
      const batch = items.slice(i, i + batchSize);
      const n = Math.floor(i / batchSize) + 1;
      console.log(` Обробка партії ${n}/${total}... `);
      const processedBatch = await processor(batch);
      results.push(...processedBatch);
    }

    return results;
  } catch (error) {
    console.error('[processInBatches] Error processing batches:', error);
    throw error;
  }
}

```

```

export async function raceWithTimeout<T>(
  promise: Promise<T>,

```

```

    timeoutMs: number
  ): Promise<T> {
    let timer: NodeJS.Timeout | undefined;

    try {
      console.log(`[raceWithTimeout] Start: setting timeout limit of ${timeoutMs}ms`);

      const timeoutPromise = new Promise<never>((_, reject) => {
        timer = setTimeout(() => {
          reject(new Error(`Operation timed out after ${timeoutMs}ms`));
        }, timeoutMs);
      });

      const result = await Promise.race([promise, timeoutPromise]);
      console.log(`[raceWithTimeout] Success: operation completed before timeout`);
      return result;
    } catch (error) {
      console.error(`[raceWithTimeout] Error/Timeout:`, error);
      throw error;
    } finally {
      if (timer) {
        clearTimeout(timer);
      }
    }
  }
}

```

### 3.2. Завдання 2: Налаштування проєкту для тестування

```

module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node'
};

```

|      |      |               |        |      |                                                 |      |
|------|------|---------------|--------|------|-------------------------------------------------|------|
|      |      | Ігнатюк А.М.  |        |      | ДУ «Житомирська політехніка».26.121.8.000 – Лр2 | Арк. |
|      |      | Фуріхата Д.В. |        |      |                                                 | 3    |
| Змн. | Арк. | № докум.      | Підпис | Дата |                                                 |      |

```
{
  "name": "nodejs",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "build": "tsc",
    "start": "tsc && node dist/index.js",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/IhnatiukAndrii/nodejs-labs.git"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/IhnatiukAndrii/nodejs-labs/issues"
  },
  "homepage": "https://github.com/IhnatiukAndrii/nodejs-labs#readme",
  "devDependencies": {
    "@types/jest": "^30.0.0",
    "@types/node": "^25.9.1",
    "jest": "^30.4.2",
    "ts-jest": "^29.4.11",
    "typescript": "^6.0.3"
  }
}
```

delay.test.ts

```
import { delay } from './delay';

describe('delay', () => {

  it('should resolve after the specified delay', async () => {

    const start = Date.now();

    await delay(100);

    const duration = Date.now() - start;

    expect(duration).toBeGreaterThanOrEqual(95);

  });

  it('should reject if negative delay is passed', async () => {

    await expect(delay(-100)).rejects.toThrow('Delay duration cannot be negative');

  });

});
```

### 3.3. Завдання 3: Тести для fetchUserProfiles

```
import { fetchUserProfiles } from './fetchUserProfiles';

describe('fetchUserProfiles', () => {

  it('should return an empty array if userIds is empty', async () => {

    const result = await fetchUserProfiles([]);

    expect(result).toEqual([]);

  });

  it('should return correctly mapped user profiles', async () => {

    const ids = ['1', '2', '3'];

    const result = await fetchUserProfiles(ids);

    expect(result).toHaveLength(3);

    expect(result).toEqual([

      { id: '1', name: 'User 1', email: 'user1@example.com' },
```

|      |      |               |        |      |                                                 |      |
|------|------|---------------|--------|------|-------------------------------------------------|------|
|      |      | Ігнатюк А.М.  |        |      | ДУ «Житомирська політехніка».26.121.8.000 – Лр2 | Арк. |
|      |      | Фуріхата Д.В. |        |      |                                                 | 5    |
| Змн. | Арк. | № докум.      | Підпис | Дата |                                                 |      |

```

    { id: '2', name: 'User 2', email: 'user2@example.com' },
    { id: '3', name: 'User 3', email: 'user3@example.com' }
  ]);
});

it('should fetch profiles in parallel', async () => {
  const ids = ['1', '2', '3'];
  const start = Date.now();
  await fetchUserProfiles(ids);
  const duration = Date.now() - start;
  expect(duration).toBeLessThan(400);
});
});

```

### 3.4. Завдання 4: Тести для retryOperation

```

import { retryOperation } from './retryOperation';

describe('retryOperation', () => {
  it('should resolve immediately if operation succeeds first time', async () => {
    const operation = jest.fn().mockResolvedValue('success');
    const result = await retryOperation(operation, 3);
    expect(result).toBe('success');
    expect(operation).toHaveBeenCalledTimes(1);
  });

  it('should retry on failure and resolve when successful', async () => {
    let attempts = 0;
    const operation = jest.fn().mockImplementation(async () => {
      attempts++;
      if (attempts < 3) {
        throw new Error('temporary error');
      }
    });
  });
});

```

```

    }

    return 'success';
  });

  const result = await retryOperation(operation, 3);
  expect(result).toBe('success');
  expect(operation).toHaveBeenCalledTimes(3);
});

it('should throw the last error if all attempts fail', async () => {
  const operation = jest.fn().mockRejectedValue(new Error('fatal error'));
  await expect(retryOperation(operation, 3)).rejects.toThrow('fatal error');
  expect(operation).toHaveBeenCalledTimes(3);
});
});

```

### 3.5. Завдання 5: Тести для processInBatches

```

import { processInBatches } from './processInBatches';

describe('processInBatches', () => {
  it('should return empty array if items is empty', async () => {
    const processor = jest.fn().mockResolvedValue([]);
    const result = await processInBatches([], 3, processor);
    expect(result).toEqual([]);
    expect(processor).not.toHaveBeenCalled();
  });

  it('should process batches sequentially and combine results', async () => {
    const items = [1, 2, 3, 4, 5];
    const processor = jest.fn().mockImplementation(async (batch: number[]) => {

```

|      |      |               |        |      |                                                 |      |
|------|------|---------------|--------|------|-------------------------------------------------|------|
|      |      | Ігнатюк А.М.  |        |      | ДУ «Житомирська політехніка».26.121.8.000 – Лр2 | Арк. |
|      |      | Фуріхата Д.В. |        |      |                                                 | 7    |
| Змн. | Арк. | № докум.      | Підпис | Дата |                                                 |      |

```

    return batch.map(n => n * 2);
  });

  const result = await processInBatches(items, 2, processor);
  expect(result).toEqual([2, 4, 6, 8, 10]);
  expect(processor).toHaveBeenCalledTimes(3);
  expect(processor).toHaveBeenNthCalledWith(1, [1, 2]);
  expect(processor).toHaveBeenNthCalledWith(2, [3, 4]);
  expect(processor).toHaveBeenNthCalledWith(3, [5]);
});

it('should map elements from one type to another', async () => {
  const items = ['a', 'bb', 'ccc'];
  const processor = async (batch: string[]): Promise<number[]> => {
    return batch.map(s => s.length);
  };
  const result = await processInBatches(items, 2, processor);
  expect(result).toEqual([1, 2, 3]);
});
});

```

### 3.6. Завдання 6: Тести для raceWithTimeout

```

import { raceWithTimeout } from './raceWithTimeout';
import { delay } from './delay';

describe('raceWithTimeout', () => {
  it('should resolve with the promise value if resolved before timeout', async () => {
    const operation = delay(50).then(() => 'success');
    const result = await raceWithTimeout(operation, 100);
    expect(result).toBe('success');
  });
});

```



```

it('should reject with a timeout error if not resolved before timeout', async () => {
    const operation = delay(200).then(() => 'success');

    await expect(raceWithTimeout(operation, 100)).rejects.toThrow('Operation timed out after 100ms');

    await operation.catch(() => {});

});
});

```

## Тестування

```

Test Suites: 5 passed, 5 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        3.908 s, estimated 4 s
Ran all test suites.

```

| File   | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|--------|---------|----------|---------|---------|-------------------|
| ...les | 87.32   | 76.92    | 100     | 86.95   |                   |
| ...ts  | 80      | 100      | 100     | 80      | 15-16             |
| ...ts  | 80.95   | 100      | 100     | 80.95   | 32-33, 42-43      |
| ...ts  | 85.71   | 100      | 100     | 84.61   | 20-21             |
| ...ts  | 93.33   | 66.66    | 100     | 93.33   | 22                |
| ...ts  | 100     | 66.66    | 100     | 100     | 5                 |

```

Test Suites: 5 passed, 5 total
Tests:       13 passed, 13 total
Snapshots:   0 total
Time:        4.688 s
Ran all test suites.

```

**Висновок:** в ході виконання роботи набув практичних навичок написання автомати-зованих тестів для асинхронного TypeScript-коду з використанням фреймворку Jest, зокрема: • налаштування Jest з TypeScript (ts-jest); • написання unit-тестів для асинхронних функцій; • використання матчерів Jest (toBe, toEqual, toThrow, toHaveBeenCalledTimes тощо); • тестування Promise-based коду через async/await та .resolves/.rejects; • використання lifecycle hooks (beforeEach, afterEach) та fake timers (jest.useFakeTimers); • вимірювання покриття коду (code coverage); • застосування generics у TypeScript для типізації тестів.

|      |      |               |        |      |                                                 |      |
|------|------|---------------|--------|------|-------------------------------------------------|------|
|      |      | Ігнатюк А.М.  |        |      | ДУ «Житомирська політехніка».26.121.8.000 – Лр2 | Арк. |
|      |      | Фвріхата Д.В. |        |      |                                                 | 10   |
| Змн. | Арк. | № докум.      | Підпис | Дата |                                                 |      |